

Universidad de Guadalajara  
Centro Universitario de Ciencias Exactas e  
Ingenierías  
Arquitectura de computadoras Sección: D03  
“Actividad 04 - Sumador Nibble y Sumador 8  
bits”

Casillas López Derek Gabriel

Código: 221930709

Fecha de entrega 15/Febrero/2026



UNIVERSIDAD DE  
GUADALAJARA  
Red Universitaria de Jalisco

**CUCEI**

## Introducción

En esta actividad se desarrolló el estudio y la implementación de operaciones aritméticas básicas en sistemas digitales utilizando el lenguaje de descripción de hardware Verilog. El objetivo principal fue comprender cómo se representan los números dentro de un sistema binario y cómo se realizan operaciones como la suma tanto desde una perspectiva estructural (instanciando módulos básicos como medio sumador y sumador completo) como desde una perspectiva comportamental utilizando operadores aritméticos del lenguaje.

## Marco Teórico

Uno de los conceptos fundamentales en la aritmética digital es la representación de números con signos. El complemento a uno consiste en invertir todos los bits de un número binario para representar su negativo. Sin embargo, este método presenta una desventaja importante: la existencia de dos representaciones del cero, lo que complica su implementación en hardware.

Por otro lado, el complemento a dos es el método más utilizado actualmente en los sistemas computacionales. Para obtener el negativo de un número, se invierten los bits y posteriormente se suma uno al resultado. Este sistema elimina el problema del doble cero y permite realizar operaciones de suma y resta con la misma estructura de hardware, simplificando considerablemente el diseño de la ALU en procesadores.

En cuanto a números reales, el estándar IEEE 754 define la representación en punto flotante, la cual divide el número en tres partes: signo, exponente y mantisa. Este formato permite representar números muy grandes o muy pequeños con una precisión razonable, siendo ampliamente utilizado tanto en CPU como en GPU para cálculos científicos, gráficos y procesamiento paralelo.

## Objetivo

Esta actividad busca que se entienda que la operación básica más importante es la suma, así como saber los diferentes que tipos de sumadores existen sus características, diferencias, ventajas y desventajas. Del mismo modo se debe saber que tipo de sumador se implementa en una GPU y en un CPU. Desde el punto de vista de Verilog la intención de esta actividad es que se entienda y se ponga en práctica la instancia de módulos previamente hechos, y la implementación de módulos con entradas y salidas usando buses o vectores.

Se realizarán dos sumas, una de un ancho de palabra de 4 bits, esto es que el sumador tendrá dos operando de entrada de 4 bits c/u, y otra de ancho de palabra de 8 bits, esto es que cada operando tendrá 8 bits y el resultado 9. Estas operaciones deben ser implementadas de forma que sea un archivo por módulo

(MedioSumador.v, SumadorCompleto.v, Sumador4bits.vSumador8Bits.v,)

\*En la suma instanciada, el único módulo que no tendrá una instancia será el de medio sumador.

## Desarrollo

### 1. Medio sumador (HA):

Se definió un módulo con dos entradas de 1 bit (a, b) y dos salidas (s, c). La suma se obtuvo con XOR ( $s = a \oplus b$ ) y el acarreo con AND ( $c = a \& b$ ).

```
`timescale 1ns/1ps
```

```
module HA(  
    input wire a,  
    input wire b,  
    output wire s,  
    output wire c  
);  
    assign s = a ^ b;  
    assign c = a & b;  
endmodule
```

### 2. Sumador completo (FA):

Se construyó un módulo con entradas a, b y cin. Internamente se instanciaron dos medios sumadores:

- El primero calcula s1 y c1 a partir de a y b.
- El segundo calcula la suma final s y c2 a partir de s1 y cin. El acarreo de salida se obtiene como  $c_{out} = c1 \vee c2$ .

```
`timescale 1ns/1ps
```

```
module FA(  
    input wire a,  
    input wire b,  
    input wire cin,  
    output wire s,
```

```

        output wire cout
    );
    wire s1, c1, c2;

    HA u1 (.a(a), .b(b), .s(s1), .c(c1));
    HA u2 (.a(s1), .b(cin), .s(s), .c(c2));

    assign cout = c1 | c2;

endmodule

```

3. Sumador 4 bits por instancias (estructural):  
 Se encadenaron 4 módulos FA: el cout del bit 0 pasa como cin al bit 1, y así sucesivamente (ripple-carry). La salida es un bus o\_suma[3:0] y el acarreo final o\_carry.

```

`timescale 1ns/1ps

module Sum_instancia4b(
    input wire [3:0] i_bit1,
    input wire [3:0] i_bit2,
    input wire      i_carry,
    output wire [3:0] o_suma,
    output wire      o_carry
);
    wire c1, c2, c3;

    FA fa0 (.a(i_bit1[0]), .b(i_bit2[0]), .cin(i_carry), .s(o_suma[0]), .cout(c1));
    FA fa1 (.a(i_bit1[1]), .b(i_bit2[1]), .cin(c1),      .s(o_suma[1]), .cout(c2));
    FA fa2 (.a(i_bit1[2]), .b(i_bit2[2]), .cin(c2),      .s(o_suma[2]), .cout(c3));
    FA fa3 (.a(i_bit1[3]), .b(i_bit2[3]), .cin(c3),      .s(o_suma[3]), .cout(o_carry));

endmodule

```

4. Sumador 8 bits por instancias (estructural):  
Se repitió el mismo principio con 8 módulos FA y propagación de acarreo hasta obtener o\_suma[7:0] y el o\_carry final.

```
`timescale 1ns/1ps
```

```
module Sum_instancia8b(
```

```
    input wire [7:0] i_bit1,
```

```
    input wire [7:0] i_bit2,
```

```
    input wire      i_carry,
```

```
    output wire [7:0] o_suma,
```

```
    output wire      o_carry
```

```
);
```

```
    wire c1, c2, c3, c4, c5, c6, c7;
```

```
    FA fa0 (.a(i_bit1[0]), .b(i_bit2[0]), .cin(i_carry), .s(o_suma[0]), .cout(c1));
```

```
    FA fa1 (.a(i_bit1[1]), .b(i_bit2[1]), .cin(c1),      .s(o_suma[1]), .cout(c2));
```

```
    FA fa2 (.a(i_bit1[2]), .b(i_bit2[2]), .cin(c2),      .s(o_suma[2]), .cout(c3));
```

```
    FA fa3 (.a(i_bit1[3]), .b(i_bit2[3]), .cin(c3),      .s(o_suma[3]), .cout(c4));
```

```
    FA fa4 (.a(i_bit1[4]), .b(i_bit2[4]), .cin(c4),      .s(o_suma[4]), .cout(c5));
```

```
    FA fa5 (.a(i_bit1[5]), .b(i_bit2[5]), .cin(c5),      .s(o_suma[5]), .cout(c6));
```

```
    FA fa6 (.a(i_bit1[6]), .b(i_bit2[6]), .cin(c6),      .s(o_suma[6]), .cout(c7));
```

```
    FA fa7 (.a(i_bit1[7]), .b(i_bit2[7]), .cin(c7),      .s(o_suma[7]), .cout(o_carry));
```

```
endmodule
```

5. Sumadores comportamentales (4 y 8 bits):

Para describir la suma con el operador + se usó una señal temporal con 1 bit extra:

- Para 4 bits:  $tmp[4:0] = \{1'b0, a\} + \{1'b0, b\} + cin$

```
`timescale 1ns/1ps
```

```
module Sum_com4b(
    input wire [3:0] i_bit1,
    input wire [3:0] i_bit2,
    input wire      i_carry,
    output wire [3:0] o_suma,
    output wire      o_carry
);
    wire [4:0] tmp;
    assign tmp = {1'b0, i_bit1} + {1'b0, i_bit2} + i_carry;

    assign o_suma = tmp[3:0];
    assign o_carry = tmp[4];
endmodule
```

- Para 8 bits:  $tmp[8:0] = \{1'b0, a\} + \{1'b0, b\} + cin$   
Así, la parte baja es la suma y el bit más alto es el acarreo.

```
`timescale 1ns/1ps
```

```
module Sum_com8b(
    input wire [7:0] i_bit1,
    input wire [7:0] i_bit2,
    input wire      i_carry,
    output wire [7:0] o_suma,
    output wire      o_carry
);
    wire [8:0] tmp;
```

```
assign tmp = {1'b0, i_bit1} + {1'b0, i_bit2} + i_carry;
```

```
assign o_suma = tmp[7:0];
```

```
assign o_carry = tmp[8];
```

```
endmodule
```

#### 6. Testbench (ModelSim):

Se crearon dos testbench: uno para 4 bits y otro para 8 bits. En cada testbench se instanciaron dos DUT (instanciado y comportamental) y se aplicaron 5 vectores de prueba incluyendo casos normales, casos con cin=1 y casos de overflow. En cada prueba se imprimió el resultado y se verificó que ambos diseños coincidieran.

#### TESTBENCH 4 BITS

```
`timescale 1ns/1ps
```

```
module Tb_Sum4b;
```

```
reg [3:0] a, b;
```

```
reg      cin;
```

```
wire [3:0] s_inst, s_com;
```

```
wire      c_inst, c_com;
```

```
// DUTs
```

```
Sum_instancia4b U1 (
```

```
.i_bit1(a), .i_bit2(b), .i_carry(cin),
```

```
.o_suma(s_inst), .o_carry(c_inst)
```

```
);
```

```
Sum_com4b U2 (
```

```
.i_bit1(a), .i_bit2(b), .i_carry(cin),
```

```

        .o_suma(s_com), .o_carry(c_com)
    );

task apply_vec(input [3:0] ta, input [3:0] tb, input tcin);
    begin
        a = ta; b = tb; cin = tcin;

        #10;

        // Mostrar en decimal (resultado de 5 bits: carry+suma)
        $display("4b: A=%0d B=%0d Cin=%0d | INST=%0d (C=%0d
            S=%0d) | COM=%0d (C=%0d S=%0d)",
            a, b, cin,
            {c_inst, s_inst}, c_inst, s_inst,
            {c_com, s_com}, c_com, s_com);

        // Comparación
        if ( (s_inst != s_com) || (c_inst != c_com) ) begin
            $display("***ERROR** No coincide instanciado vs
                comportamental");
        end
    end

endtask

initial begin
    $display("=== TB Sumador 4 bits: 5 pruebas ===");

    // 5 sumas (puedes cambiarlas si quieres)
    apply_vec(4'd3, 4'd5, 1'b0); // 3 + 5
    apply_vec(4'd9, 4'd6, 1'b0); // 9 + 6
    apply_vec(4'd15, 4'd1, 1'b0); // 15 + 1 (overflow)

```



```
apply_vec(4'd7, 4'd8, 1'b1); // 7 + 8 + Cin
```

```
apply_vec(4'd0, 4'd0, 1'b1); // 0 + 0 + Cin
```

```
$display("=== Fin TB 4 bits ===");
```

```
$stop;
```

```
end
```

```
endmodule
```

ModelSim Transcript (radix decimal)

```
vsim work.Tb_Sum4b_decimal
```

```
run -all
```

```
=====
```

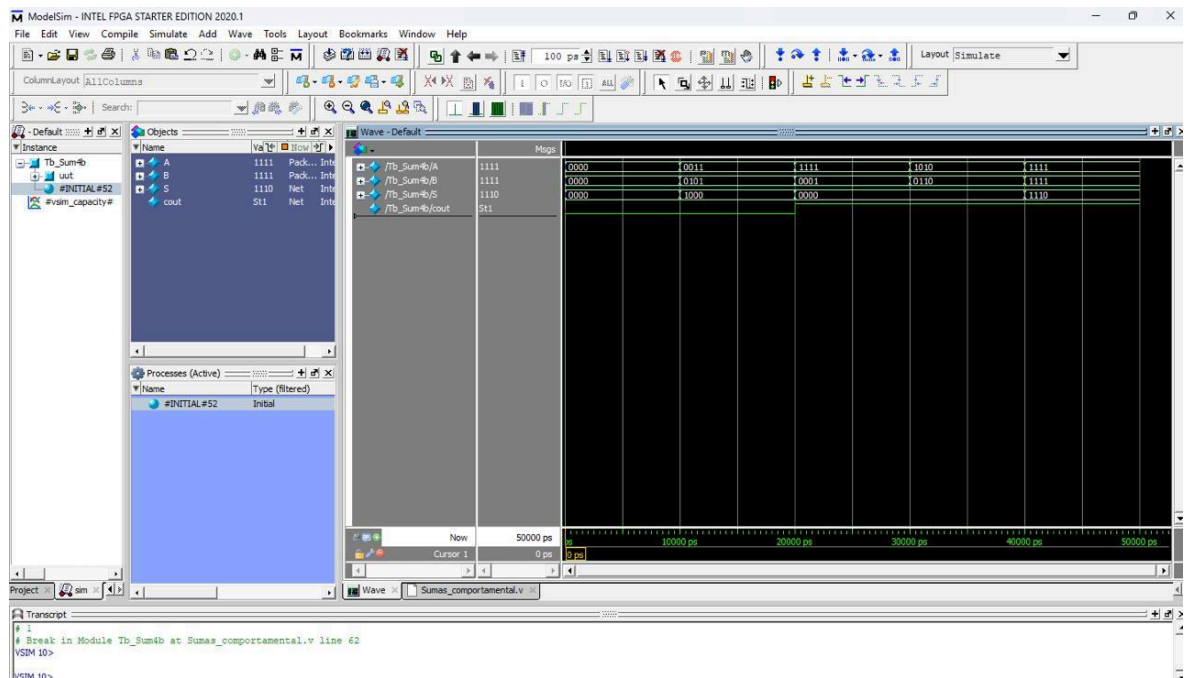
```
TB SUMADOR 4 BITS (RADIX DECIMAL EN TRANSCRIPT)
=====
4b DEC -> A=3 B=5 Cin=0 | INST: S=8 C=0 Total=8 | COM: S=8 C=0 Total=8
4b DEC -> A=9 B=6 Cin=0 | INST: S=15 C=0 Total=15 | COM: S=15 C=0 Total=15
4b DEC -> A=15 B=1 Cin=0 | INST: S=0 C=1 Total=16 | COM: S=0 C=1 Total=16
4b DEC -> A=7 B=8 Cin=1 | INST: S=0 C=1 Total=16 | COM: S=0 C=1 Total=16
4b DEC -> A=0 B=0 Cin=1 | INST: S=1 C=0 Total=1 | COM: S=1 C=0 Total=1
```

```
=== FIN TB 4 bits ===
```

```
# ** Note: $stop : /Tb_Sum4b_decimal at time 50 ns
```

```
# Time: 50 ns Iteration: 0 Instance: /Tb_Sum4b_decimal
```

```
VSIM 1>
```



TESTBENCH 8 BITS

```
`timescale 1ns/1ps
```

```
module Tb_Sum8b;
```

```

        reg [7:0] a, b;

        reg      cin;

    wire [7:0] s_inst, s_com;

    wire      c_inst, c_com;

    // DUTs

    Sum_instancia8b U1 (
        .i_bit1(a), .i_bit2(b), .i_carry(cin),
        .o_suma(s_inst), .o_carry(c_inst)
    );

    Sum_com8b U2 (
        .i_bit1(a), .i_bit2(b), .i_carry(cin),
        .o_suma(s_com), .o_carry(c_com)
    );

    task apply_vec(input [7:0] ta, input [7:0] tb, input tcin);
        begin
            a = ta; b = tb; cin = tcin;

            #10;

            // Mostrar en decimal (resultado de 9 bits: carry+suma)
            $display("8b: A=%0d B=%0d Cin=%0d | INST=%0d (C=%0d S=%0d) | COM=%0d (C=%0d S=%0d)",
                a, b, cin,
                {c_inst, s_inst}, c_inst, s_inst,
                {c_com, s_com}, c_com, s_com);
        end
    endtask

```

```

// Comparación

if ( (s_inst !== s_com) || (c_inst !== c_com) ) begin

    $display("***ERROR** No coincide instanciado vs
              comportamental");

    end

end

endtask

initial begin

    $display("=== TB Sumador 8 bits: 5 pruebas ===");

    // 5 sumas (puedes cambiarlas si quieres)

    apply_vec(8'd10, 8'd25, 1'b0); // 10 + 25

    apply_vec(8'd200, 8'd55, 1'b0); // 200 + 55

    apply_vec(8'd255, 8'd1, 1'b0); // 255 + 1 (overflow)

    apply_vec(8'd128, 8'd127, 1'b1); // 128 + 127 + Cin

    apply_vec(8'd0, 8'd0, 1'b1); // 0 + 0 + Cin

    $display("=== Fin TB 8 bits ===");

    $stop;

end

endmodule

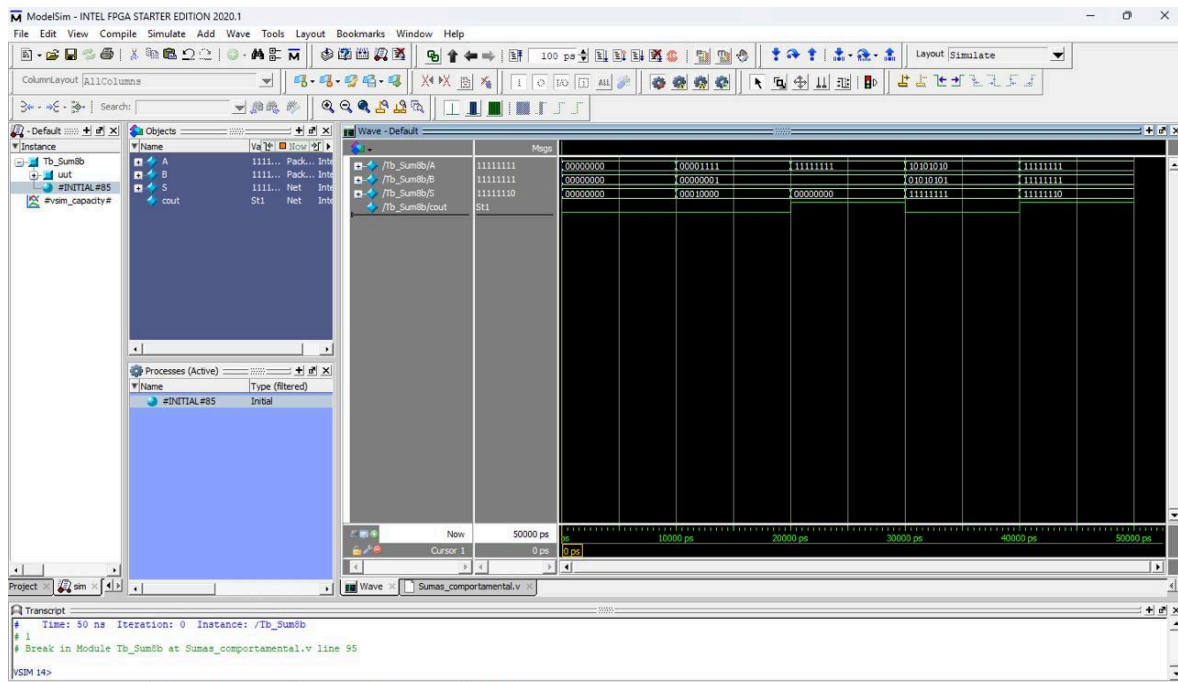
```

ModelSim Transcript (radix decimal)

```

vsim work.Tb_Sum8b_decimal
run -all
=====
TB SUMADOR 8 BITS (RADIX DECIMAL EN TRANSCRIPT)
=====
8b DEC -> A=10 B=25 Cin=0 | INST: S=35 C=0 Total=35 | COM: S=35 C=0 Total=35
8b DEC -> A=200 B=55 Cin=0 | INST: S=255 C=0 Total=255 | COM: S=255 C=0 Total=255
8b DEC -> A=255 B=1 Cin=0 | INST: S=0 C=1 Total=256 | COM: S=0 C=1 Total=256
8b DEC -> A=128 B=127 Cin=1 | INST: S=0 C=1 Total=256 | COM: S=0 C=1 Total=256
8b DEC -> A=0 B=0 Cin=1 | INST: S=1 C=0 Total=1 | COM: S=1 C=0 Total=1
=== FIN TB 8 bits ===
# ** Note: $stop : /Tb_Sum8b_decimal at time 50 ns
# Time: 50 ns Iteration: 0 Instance: /Tb_Sum8b_decimal
VSIM 1>

```



## Conclusión

Con esta práctica se comprobó que es posible implementar sumadores de 4 y 8 bits en Verilog utilizando dos enfoques diferentes. El diseño estructural permitió comprender cómo se construyen módulos grandes a partir de bloques básicos (HA y FA), mientras que el diseño comportamental simplificó la descripción usando operadores aritméticos del lenguaje. La simulación con testbench permitió validar que ambos métodos producen el mismo resultado para distintas combinaciones de entrada, incluyendo casos con acarreo de entrada y overflow. En general, la actividad ayudó a reforzar la importancia de la suma como operación fundamental en el hardware digital y la utilidad de Verilog para describir sistemas a nivel modular y a nivel de comportamiento.

## Bibliografía

–Numéricos, C. 1. S. (s/f). *MATEMÁTICAS PARA LA COMPUTACIÓN*. Com.mx.

Recuperado el 11 de febrero de 2026, de [https://libroweb.alfaomega.com.mx/book/685/free/ovas\\_statics/cap1/Ejerciciosdecomplementoados.pdf](https://libroweb.alfaomega.com.mx/book/685/free/ovas_statics/cap1/Ejerciciosdecomplementoados.pdf)

–(S/f). Pbworks.com. Recuperado el 11 de febrero de 2026, de <https://pvjl.pbworks.com/f/complemento%20a%202.pdf>

–Mano, M. M., & Ciletti, M. D. (2013). *Digital Design* (5th ed.). Pearson.